

OpenVAF-r Built-in Natures and Disciplines

The Accellera standard natures and disciplines shipped in the openvaf-r compiler

Ngspice + OpenVAF Enhancements project

July 2026

Contents

OpenVAF-r built-in natures and disciplines	2
Where they come from	3
Disciplines	4
Natures	5
How the pieces are used	6
Discrete disciplines and the digital caveat	7
User-defined natures and disciplines	8
Quick reference: which access function goes with which discipline	9

OpenVAF-r built-in natures and disciplines

Every Verilog-A net has a discipline, and every discipline binds one or two natures — one for its *potential* (across) signal and one for its *flow* (through) signal. A nature fixes the physical units, the access function you use to read/contribute the signal (`V(...)`, `I(...)`, `Pos(...)`, ...), and the default absolute tolerance the solver applies to it.

`openvaf-r` ships the complete Accellera standard set built in — 16 natures and 11 disciplines — so a model can ``include "disciplines.vams"` (or rely on the compiler resolving it) and use any of them without providing a header. This page catalogs exactly what is available and how the pieces relate.

Companion reading: the [OpenVAF compiler internals](#) guide (how the pipeline works) and the language-coverage audit against *Practical Guide to Verilog-A*.

Where they come from

The standard header is embedded in the compiler — it is not read from disk. `openvaf-r` registers a virtual `/std/disciplines.vams` (and `constants.vams`) from a string constant compiled into the binary, so any of these spellings resolves to the built-in copy:

```
`include "disciplines.vams"    // also: disciplines.va, disciplines.h, discipline.h
`include "constants.vams"     // also: constants.va, constants.h
```

The content is Accellera Verilog-AMS `disciplines.vams`, version 2.4.0, verbatim. Source in the tree: [openvaf/vfs/src/va_std.rs](#) (`DISCIPLINES_SRC`).

Disciplines

A conservative discipline has both a potential and a flow nature and obeys Kirchhoff's laws (KCL on the flow, KVL on the potential). A signal-flow discipline carries only one of the two. A discrete discipline is for digital nets (see the caveat below).

Discipline	Kind	Potential (access)	Flow (access)	Domain
electrical	conservative	Voltage — V	Current — I	continuous
voltage	signal-flow	Voltage — V	—	continuous
current	signal-flow	—	Current — I	continuous
magnetic	conservative	Magneto_Motive_Force — MMF	Flux — Φ	continuous
thermal	conservative	Temperature — Temp	Power — Pwr	continuous
kinematic	conservative	Position — Pos	Force — F	continuous
kinematic_v	conservative	Velocity — Vel	Force — F	continuous
rotational	conservative	Angle — Theta	Angular_Force — Tau	continuous
rotational_omega	conservative	Angular_Velocity — Ω	Angular_Force — Tau	continuous
\logic	discrete	—	—	discrete
ddiscrete	discrete	—	—	discrete

\logic is an escaped identifier (the name is logic); use it as \logic with the trailing space.

Natures

Each nature is grouped below by physical domain. `access` is the function name you call to read or contribute the signal; `abstol` is the default absolute tolerance (overridable — see below); `ddt_nature` / `idt_nature` name the nature that a time derivative / time integral of this signal takes.

Electrical

Nature	Units	Access	abstol	ddt_nature	idt_nature
Current	A	I	1e-12	—	Charge
Charge	coul	Q	1e-14	Current	—
Voltage	V	V	1e-6	—	Flux
Flux	Wb	Phi	1e-9	Voltage	—

Magnetic

Nature	Units	Access	abstol	ddt_nature	idt_nature
Magneto_Motive_Force	A*turn	MMF	1e-12	—	—

(Magnetic flux uses the electrical `Flux` nature above.)

Thermal

Nature	Units	Access	abstol	ddt_nature	idt_nature
Temperature	K	Temp	1e-4	—	—
Power	W	Pwr	1e-9	—	—

Kinematic (translational)

Nature	Units	Access	abstol	ddt_nature	idt_nature
Position	m	Pos	1e-6	Velocity	—
Velocity	m/s	Vel	1e-6	Acceleration	Position
Acceleration	m/s^2	Acc	1e-6	Impulse	Velocity
Impulse	m/s^3	Imp	1e-6	—	Acceleration
Force	N	F	1e-6	—	—

Rotational

Nature	Units	Access	abstol	ddt_nature	idt_nature
Angle	rads	Theta	1e-6	Angular_Velocity	—
Angular_Velocity	rads/s	Omega	1e-6	Angular_Acceleration	Angle
Angular_Acceleration	rads/s^2	Alpha	1e-6	—	Angular_Velocity
Angular_Force	N*m	Tau	1e-6	—	—

How the pieces are used

Access functions. In a module you name the discipline on a net, then read and contribute signals through the nature's access function. For a conservative discipline the two-argument form is a branch across a node pair; the one-argument form is relative to ground:

```
electrical p, n;
analog begin
    I(p, n) <+ V(p, n) / R;           // Ohm's law on the p-n branch (flow = Current,
↪  access I)
    I(p, n) <+ C * ddt(V(p, n));     // capacitive current on the same branch
end
```

You contribute to a branch through its flow access function (I for electrical); the potential accessor (V) is used to *read* the across signal. The Charge nature (access Q) is not the flow of any standard discipline — it appears only as Current's `idt_nature` — so charge storage is modelled as a current, `I(...) <+ ddt(Q_expr)`, rather than contributed directly.

`abstol` and per-model overrides. The tolerances above are defaults. Define the matching `<NATURE>_ABSTOL` macro *before* including the header to override one, e.g.

```
`define VOLTAGE_ABSTOL 1e-9
`include "disciplines.vams"
```

The recognized macros are ``CURRENT_ABSTOL`, ``CHARGE_ABSTOL`, ``VOLTAGE_ABSTOL`, ``FLUX_ABSTOL`, ``MAGNETO_MOTIVE_FORCE_ABSTOL`, ``TEMPERATURE_ABSTOL`, ``POWER_ABSTOL`, ``POSITION_ABSTOL`, ``VELOCITY_ABSTOL`, ``ACCELERATION_ABSTOL`, ``IMPULSE_ABSTOL`, ``FORCE_ABSTOL`, ``ANGLE_ABSTOL`, ``ANGULAR_VELOCITY_ABSTOL`, ``ANGULAR_ACCELERATION_ABSTOL`, and ``ANGULAR_FORCE_ABSTOL`.

`ddt_nature` / `idt_nature`. These chain a signal to the nature of its time derivative or integral, so `ddt()` / `idt()` inherit a sensible tolerance. For example `Charge.ddt_nature = Current` means $d(\text{charge})/dt$ is treated as a current; `Voltage.idt_nature = Flux` means the time integral of a voltage is a flux. The translational and rotational families form full chains (position $\rightarrow$ velocity $\rightarrow$ acceleration $\rightarrow$ impulse, and the angular analogues).

Discrete disciplines and the digital caveat

`\logic` and `ddiscrete` are present so that standard headers referencing them parse, but they are `do-main discrete` (digital) disciplines with no potential or flow nature. `openvaf-r` is an analog Verilog-A compiler: it does not support digital/discrete *signal flow* (it rejects `wreal` nets outright). So while these two disciplines exist in the header, you cannot build a digital behavioral model with them — use the continuous, conservative disciplines above. This matches the language-coverage audit's "mixed-signal is out of scope by design" finding.

User-defined natures and disciplines

The built-in set is not a closed list. A model may define its own nature (with units / access / abstol, optionally deriving from a base nature) and its own discipline (binding a potential and/or flow), at the top level of the file — `openvaf-r` compiles these normally. This is how domain-specific models add, say, a chemical-concentration or optical net-discipline that the standard header does not provide.

```
nature Concentration;  
    units  = "mol/m^3";  
    access = CH;  
    abstol = 1e-6;  
endnature  
discipline chemical_sf;  
    potential Concentration;  
enddiscipline
```

Quick reference: which access function goes with which discipline

If you declare a net...	read potential with	read flow with
electrical	V(...)	I(...)
magnetic	MMF(...)	Phi(...)
thermal	Temp(...)	Pwr(...)
kinematic	Pos(...)	F(...)
kinematic_v	Vel(...)	F(...)
rotational	Theta(...)	Tau(...)
rotational_omega	Omega(...)	Tau(...)
voltage (signal-flow)	V(...)	—
current (signal-flow)	—	I(...)

Authoritative source: the embedded Accellera disciplines.vams v2.4.0 in [openvaf/vfs/src/va_std.rs](https://github.com/accellera/opensvdf/blob/master/src/va_std.rs). Every discipline and nature above was cross-checked by compiling a probe module per discipline with the shipped `openvaf-r`.